Transforming Software Development: A Study on the Integration of Multi-Agent Systems and Large Language Models for Automatic Code Generation

Rolando Ramírez-Rueda
Facultad de Estadística e Informática
Universidad Veracruzana
Xalapa, Mexico
zS20000354@estudiantes.uv.mx

Edgard Benítez-Guerrero
Facultad de Estadística e Informática
Universidad Veracruzana
Xalapa, México
edbenitez@uv.mx

Carmen Mezura-Godoy
Facultad de Estadística e Informática
Universidad Veracruzana
Xalapa, México
cmezura@uv.mx

Everardo Bárcenas
Facultad de Ingeniería
Universidad Nacional Autónoma de México
Ciudad de México, México
barcenas@fi-b.unam.mx

***Abstract*—This paper explores the integration of Multi-Agent Systems (MAS) and Large Language Models (LLMs) for automatic code generation, addressing the limitations of traditional manual coding. By conducting a comprehensive review of existing literature and analyzing a practical case study, we demonstrate how MAS and LLMs can collaboratively enhance software development processes. The research focuses on the technical and theoretical challenges of this integration, highlighting the potential for improved productivity, adaptability, and quality in code generation. The findings contribute to AI-based software engineering by revealing new research directions in collective intelligence and automated programming.**

***Keywords*-Multi-Agent, Generative AI, LLM, Software Development**

I. Introduction

During software development, the coding phase usually involves human manual processes [1]. Program synthesis has emerged as an alternative approach, where the code is automatically generated from an expression of user needs. A successful synthesis of programs would not only improve the productivity of experienced programmers but also make programming accessible to non-experts [2].

Traditionally, program synthesis has been approached using a variety of methods, including rule-based systems. Although these approaches have proven to be useful, they face limitations in terms of flexibility, scalability, and adaptability to new domains or programming languages [3].

With the emergence of Large Language Models (LLMs), a revolution has been witnessed in automatic code generation [4]. These models, trained on large corpus of source code, are capable of generating code from natural language descriptions, and also offer the potential to understand more complex contexts and perform more sophisticated coding tasks. However, effectively integrating LLMs into software development environments still presents challenges, especially with regards to customization and extension to meet project or domain-specific needs [5], [6].

In this context, the proposal of multi-agent systems (MAS) emerges as a promising solution, because these systems can contemplate intelligent agents, each with specific roles and responsibilities, which allow customizable and extensible aspects [7]. This approach not only facilitates the adaptation of LLMs to different software development contexts, but it also opens an ideal scenario to study collective intelligence between different agents in code generation. Another important point of multi-agent systems is that they can improve the efficiency and quality of the generated code, dynamically adapting to changing users needs [8] [7].

This article aims to explore the feasibility of using multi-agent systems and LLMs to mitigate the limitations of traditional manual code creation methods, thinking that in the future this process will be more easy, customizable and extensible [9]. Let us note that [10] discusses a broad vision of the interactions of agents and GPTs for software enginering tasks, but we take a step forward. By reviewing the state of the art and exploring this innovative approach, we seek to contribute to the field of AI-based software engineering and open new avenues of research in automatic code generation and collective intelligence. In doing so, we will address the theoretical and technical challenges involved in this integration, with the ultimate goal of facilitating software development and enriching our understanding of the collaborative capabilities between agents and large-scale language models.

The remainder of this paper is organized as follows. First, background on the relevant research areas of Multi-Agent Systems and LLMs is presented. Then, related works are discussed. The research method we followed is presented

next, and then the results obtained are discussed. Finally, conclusions drawn from this research are presented.

II. Background

In this section, we provide the necessary background on multi agent systems and large language models for Software Development.

A. Multi-agent Systems for Software Development

Multi-agent systems emerges as a promising strategy to address the challenges and limitations in automatic code generation. A multi-agent system (MAS) is a framework where multiple autonomous agents interact with each other. These agents, which may be program scripts, software robots, or other purpose-programmed software, operate in a shared environment and may communicate, cooperate, compete, or negotiate with each other. Each agent in a multi-agent system has its own capabilities, goals, and perceptions, and works independently or together to achieve complex goals or solve problems [7].

In a multi-agent system, each agent can be assigned a specialized role, which facilitates the distribution of complex tasks into more manageable subtasks. For example, one agent may be responsible for understanding user specifications, another agent may generate the code base, while other agents may improve the generated code, optimize it, and perform quality testing. This distribution of roles allows for efficient division of labor and collaboration between agents, with the aim of increasing the efficiency and quality of the software development process [9].

Multi-agent systems are inherently adaptive and customizable, making them ideal for changing software development environments [11]. Each agent can be configured and trained to fit the specific needs of a project or application domain. Furthermore, agents can continually learn and improve by interacting with the environment and exchanging knowledge with other agents. This ability to adapt and learn ensures that the multi-agent system evolves over time and remains relevant in dynamic software development contexts.

As seen, the use of multi-agent systems allows collaboration and the management of the inherent complexity of automatic code generation, especially when working with large volumes of code or complex requirements. Agents can divide and solve complex problems considering that each agent will have a domain of knowledge, using coordination and communication strategies to integrate partial solutions into a coherent global solution. Additionally, multi-agent systems are inherently scalable, meaning they can efficiently handle large data sets and operate in distributed environments without compromising performance or quality of the end result.

Multi-agent systems encourage collective intelligence and diversity of approaches in code generation. Each agent can bring unique perspectives and problem-solving strategies, thus enriching the code generation process with a variety of approaches and alternative solutions [12]. Additionally, agent diversity can help mitigate biases and limitations, thus promoting more balanced and robust code generation. In summary, multi-agent systems offers an innovative and promising approach to improve automatic code generation.

B. Large Language Models for Software Development

Large Language Models have revolutionized automatic code generation in recent years. This type of capability is especially useful in scenarios where rapid prototyping or automation of repetitive tasks is required. LLMs can adapt to different coding styles and different programming languages, making them very versatile and applicable to a wide range of software development projects [13].

Despite these advances, LLM-based code generation present significant challenges, such as the need to ensure that the generated code is not only functional, but also efficient, effective and usable. For example, LLMs may generate code that meets functional requirements, but may not consider performance or usability considerations, which can lead to problems in the implementation of the final software.

Another important aspect is that, to generate accurate code, the LLM must have a clear understanding of the context. As a result, this approach to code generation can face difficulties when the LLM encounter ambiguous instructions or complex contexts, which can result in the generation of incorrect or inefficient code [14]. Therefore, it is necessary to constantly improve training methods and adjust the models, as well as to propose complementary techniques to verify and validate the generated code.

Finally, LLM-based code generation offers the promise of simplifying and accelerating software development. The integration of LLMs into software development environments requires addressing different technical challenges since LLMs have the potential to fundamentally transform the way software is developed, allowing programmers to focus on higher-level tasks and encouraging innovation in the field of software engineering.

III. Related Works

The integration of LLMs within multi-agent collaboration systems represents a cutting-edge technology [15]. By combining the power of LLMs with the adaptability and collaboration of multi-agent systems, it is possible to open new frontiers in automatic code generation and collective intelligence in software development.

This Section presents an analysis of the state of the art on automatic code generation using Multi-Agent Systems and LLMs. We followed the systematic literature review method proposed by [16], in addition to considering the recommendations given by [17], [18]. To complement this review, we include in the discussion other studies have not been subjected to peer review; however, they provide

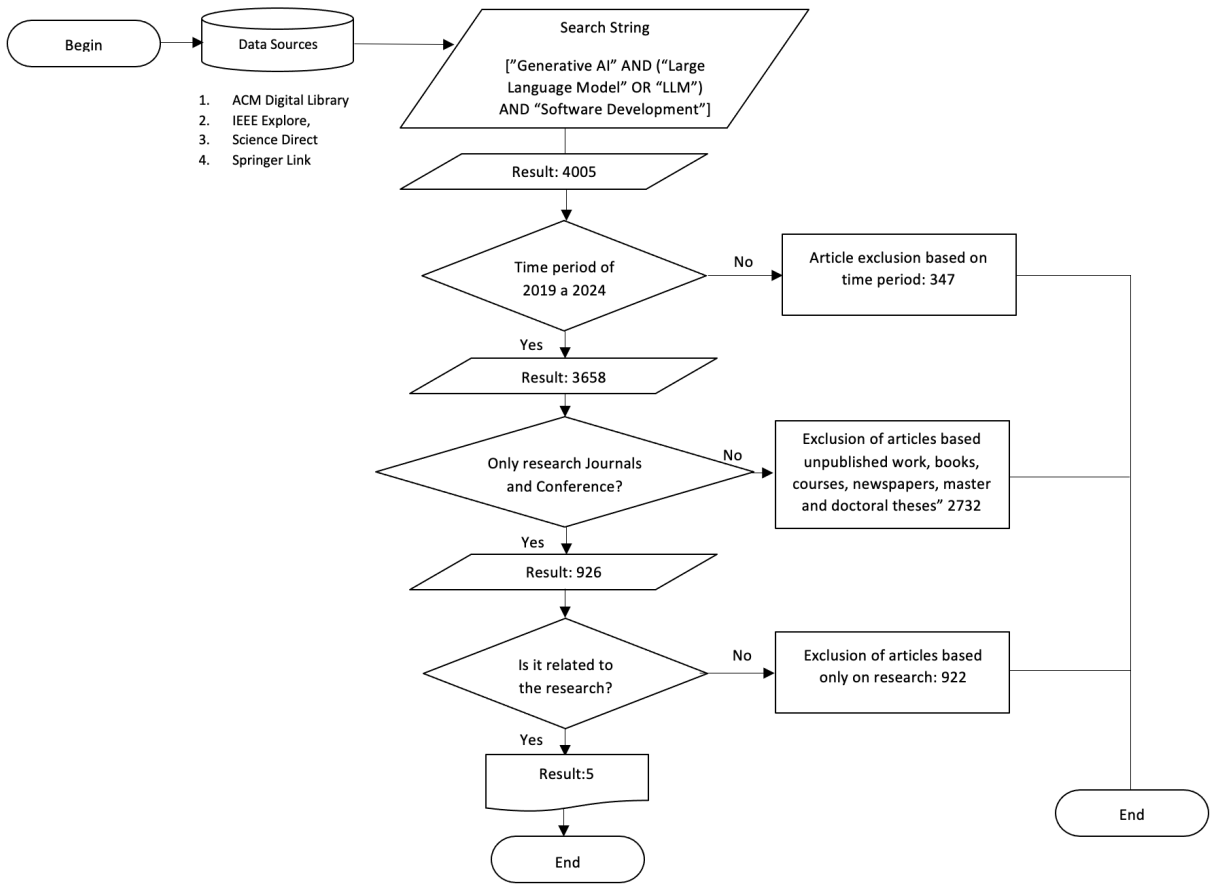

Fig. 1. Steps for the extraction of relevant documents from the selected sources

information on the current direction of the research and we believe that it is appropriate to consider their points of view as a reference for future work.

A. Planning

1) Research Questions: The aim of our systematic literature review is to comprehensively understand how multi-agent systems are being used for software development, explore how they work with large language models, and identify areas that deserve further investigation. The research questions for this study were defined as follows:

- Q1: Are there any works on LLM-based multi-agent systems for software development?
- Q2: If so, How are those MAS implemented?
- Q3: What LLMs are used and what types of prompts are used?
- Q4: What are the advantages and challenges of each work?

2) Search String: In order to define the search string, we initially identified that when searching for "Multi-Agent System" and "Large Language Model" the results were very limited. Thus we decided to modify it, eliminating the keyword "Multi-Agent System" and replaced it with "Generative AI", that give us results considering the notion of "Agent". Finally, we included "Software Development". Therefore the final search string was the following: ["Generative AI" AND ("Large Language Models OR "LLM") AND "(Software Development")].

3) Data Sources: Four major data sources were selected to conduct a comprehensive search for literature relevant to our research. The sources were ACM Digital Library, IEEE Xplore, Springer, and Elsevier. These platforms were chosen due to their extensive repositories of scientific papers and their relevance to the fields under study.

4) Selection Criteria: To ensure focused and relevant data collection, several inclusion and exclusion criteria were considered:

- Inclusion Criteria: Only journal and conference papers published from 2019 to 2024 were considered. This allowed us to capture the most recent advances in the field. The search parameters were meticulously established to filter data by titles, abstracts, and keywords that meet the inclusion criteria.
- Exclusion criteria: Unpublished works, books, courses, newspapers, and master's and doctoral degrees theses were excluded from the search.

These criteria facilitated the collection of the most relevant papers related to our research.

5) Quality assessment: Each study was evaluated using the criteria of the Center for Reviews and Dissemination (CDR) of the University of York, as well as the Database of Abstracts of Reviews of Effects (DARE) [19]. The criteria are based on three quality assessment (QA) questions:

- QA1. Are sufficient details about the studies presented?
- QA2. Does it provide evidence to answer the research questions for this systematic review?
- QA3. Is it a referenced study?

The questions were scored as follows:

- QA1: Y (yes), the paper presents sufficient details; P (Partially), it presents information partially; N (no), it does not have details and they cannot be easily inferred.
- QA2: Y (yes), the paper included appropriate strategies, identified and made reference to all the journals that addressed the topic of interest, and the study answered all the research questions; P (Partially), the paper only partially answered the research questions; N (no), the paper did not answer the research questions, and lacked adequate context.
- QA3: Y (yes), It is a highly referenced study; P (Partially), the study has a certain number of citations; N (no), the study does not have citations.

The scoring procedure was Y = 1, P = 0.5 and N = 0. Consequently, the possible score that a paper could obtained for assessing its quality was in the range of 0 to 3 points. In this sense, the papers considered had to achieve a rating of 1.5 at least.

B. Quantitative results

The methodology applied in this search is summarized in Fig. 1. The search across the specified data sources initially yielded a total of 4005 articles. By applying the inclusion criteria of publication years from 2019 to 2024 and focusing on journal articles and conference proceedings, the results were refined to 926 articles. Further application of criteria related to the research topic narrowed this down to 4 articles that were directly relevant to the research objectives.

Among these 4 pertinent articles, 2 (50%) were published in journals, while 2 (50%) appeared in conference proceedings, as depicted in Fig. 2. We observed a clear trend of increasing publications up until 2023. This trend could potentially increase in 2024, influenced by emerging developments in the field.

The research questions and corresponding answers presented in this study have significant implications. Only 4 papers were directly relevant to the research questions under consideration, as detailed in Table I.

The results of the application of the quality evaluation show that, on average, the studies had a score of 2.125 points, with the exception of study S1 that obtained a score of 1.5. As it can be seen in Table II, the articles are of good quality and relevant to our research.

TABLE I
TOTAL ITEMS EXTRACTED.

Data Sources	Result	Useful Articles	Accuracy
IEEE	23	3	0.1304%
ACM	1855	1	0.0005%
SPRINGER	753	0	0%
ELSEVIER	1027	0	0%

TABLE II
EVALUATION OF THE QUALITY OF THE STUDIES.

Study	QA1	QA2	QA3	Total Score
S1 [20]	P	Y	N	1.5
S2 [21]	Y	Y	N	2
S3 [22]	Y	Y	P	2.5
S4 [23]	Y	Y	P	2.5

C. Qualitative results

1) LLM-based multi-agent systems for software development (Q1): The work [20] proposes the use of autonomous agents equipped with LLM to carry out diverse tasks within the Software Development Life Cycle (SDLC), such as code generation, debugging, test case generation and script execution. Auto-GPT, a tool that incorporates these agents, acts as a personal assistant for developers, offering a wide range of functionalities without the need for continuous human interaction.

The study [21] examines critical aspects in the generation of functional and error free applications, and also addresses aspects such as project planning, including scope preparation, schedule development, cost estimation, resource evaluation, quality, stakeholder mapping, communication, planning and risk analysis. The results indicate unique strengths and weaknesses for software generated by AI and generated by humans. Therefore, a normative and adaptive multi-agent system is proposed that uses LLM to improve the capacity of agents to interpret and comply with regulations within their environment. This system is designed to allow agents to respond in a flexible and adaptive manner to regulatory changes and environmental conditions.

The system proposed in [22] is an adaptive multi-agent system with the objective of future investigations into the modeling of adaptive agents capable of understanding and dealing with dynamic problems that help developers model and create software. In this case, a new metamodel is used to help agents adapt and respond to norms within their environment. This approach allows agents to adapt their behavior as the conditions and norms that regulate their behavior change.

Although [23] does not directly deals with software generation as such, it proposes a novel framework for the detection of vulnerabilities in smart contracts involved in its construction, using Large Language Models, through a

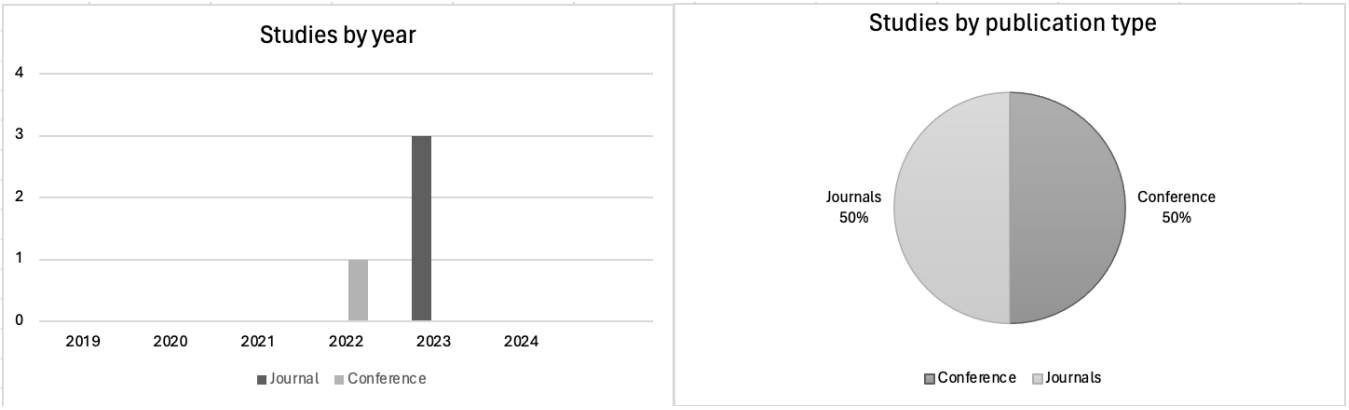

Fig. 2. Type of articles according to inclusion criteria and average of article according to year of publication.

two-step adversarial detection approach called GPTLENS. This system aims to detect vulnerabilities, achieve usability aspects by identifying as many true vulnerabilities as possible and at the same time minimizing the number of false positives using agents with dual LLM roles such as Auditor and Critical.

2) How the system is implemented (Q2): Autonomous Orchestration in Software Development Life Cycle [20] uses the GPT-3.5 and GPT-4 models [24], model to perform tasks autonomously based in Auto-GPT. The system is organized around a workflow where autonomous agents manage assigned tasks: a task agent creates and manages a list of tasks, a prioritization agent assigns priorities, and an execution agent carries out the tasks using contexts recovered in the agent's memory. This process ends with the delivery of results to the user.

The system [21] is organized using a new metamodel that facilitates the adaptation of agents to dynamic and changing norms. This metamodel integrates principles of artificial intelligence, modeling agents and normative systems, providing a structure that allows agents to interpret norms and adapt their behavior effectively.

In [22], the authors proposed a system that is organized using a new metamodel that integrates adaptive and normative concepts into the design and execution of multi-agent systems. The metamodel allows agents to understand and adapt to norms addressed to them in their environment, using a specific modeling language developed to explicitly represent these concepts.

The GPTLENS system [23] organizes vulnerability detection in two adversarial stages: Generation stage and Discrimination stage. In the former, multiple Auditor agents generate responses that identify possible vulnerabilities with a high diversity of responses, while in the latter, a critical agent evaluates the validity of the vulnerabilities identified in the first stage, classifying the responses according to criteria of correction, severity and profitability.

3) LLMs used and the types of promts used (Q3): The work [20] uses GPT-3.5 and GPT-4 models [24]. The types of prompts that are used include both high-level context-free prompts and context-rich prompts that allow context-based prompts to be created, as they have been shown to be more effective in understanding user requirements and avoiding irrelevant details that could affect the understanding of the task and degrade the performance of the model.

The system [21] uses LLMs to process and understand regulations and instructions within the environment in which agents operate. Although the document does not specify a particular LLM model, it focuses on the capacity of these models to understand and generate language that corresponds to relevant standards and guidelines. The indications used include normative guidelines and behavior protocols that agents need to follow.

The paper [22] does not specify the use of a specific Large Language Model, nor detail about the indications used for communication or the language process in this context. The main focus is on the development of the metamodel and the model language for the adaptation and regulation of agents.

The system [23] uses GPT-4 for both functions, defining an Auditor and a Critic. The Auditor is required to identify and explain the detected vulnerabilities, including their reasoning and association with the specific code. The Critic must evaluate the vulnerabilities identified by the Auditor, to do so he must assign scores based on the criteria of correctness, severity and profitability.

4) Advantages and Challenges (Q4): The use of Auto-GPT [20] and similar agents represents a significant advance in software engineering, offering the possibility of automating and optimizing numerous SDLC tasks. However, the effectiveness of these systems depends to a large extent on the quality and context of the indications provided.

Advantages:

- Task automation in SDLC: Auto-GPT can independently manage multiple aspects of software development, reducing the need for human intervention.

- Improves requirements understanding: Indications in context significantly improve the understanding of user requirements and the generation of coherent code.

Challenges:

- Complex task management: Needs accurate contextual information to perform complex operations correctly.
- Hallucinations: Generation of classes, unnecessary methods or variables, the creation of files on incorrect routes.

The approach presented in [21] represents a significant integration of LLM technology into multi-agent systems, offering new possibilities for adaptation and normative conformity in dynamic environments.

Advantages:

- Adaptability: Agents can adapt quickly to new norms and changes in the environment, which improves the efficiency of the system.
- Improved interpretation of standards: LLMs can help agents understand and apply complex standards, which can improve the coherence and efficiency of agent.

Challenges:

- Implementation complexity: Integrating LLMs into regulatory multi-agent systems can be complex due to the need to constantly adapt standards and guidelines.
- Interpreting standards: Ensuring that agents interpret and apply standards correctly can be difficult, especially in environments with rapidly changing standards.

The paper [22] proposes an advanced approach to the modeling and execution of multi-agent systems that can adapt and operate within regulatory frameworks, which is crucial for applications in regulated and constantly changing environments.

Advantages:

- Improved Adaptability: Agents can adapt their behavior more effectively to changing conditions and environmental norms.
- Behavior Control: Standards provide a method to regulate the behavior of agents in a way that aligns with the objectives of the system.

Challenges:

- Complexity in the model: Incorporating adaptations and regulations into the model of multi-agent systems presents challenges in terms of the complexity of the design and execution.
- Implementation of Standards: Ensuring that agents understand and adapt correctly to standards can be complicated, especially in dynamic environments.

GPTLENS [23] represents an innovative application of LLMs in the detection of smart contract vulnerabilities, offering a detailed and strategically organized approach to improve efficiency and accuracy in this field.

Advantages:

- Improves detection accuracy: By dividing detection into two phases and specializing LLM roles, accuracy improves in identifying real vulnerabilities.
- Generalization: The system has the potential to identify a wider range of types of vulnerabilities, including those unknown or not previously categorized.

Challenges:

- Management of false positives and negatives: Although the model seeks to minimize false positives, the generation of a large number of responses can increase their number, as well as the number of false negatives if vulnerabilities are not detected due to randomness in response generation.
- Computational costs: The operation of a GPT-4-based system for multiple agents is computationally intensive.

D. Discussion of related works

The findings of this review suggest that as generative tools for user-centered design continue to advance in functionality and communication capabilities, their contribution to different areas of computing will become increasingly significant.

Let us note that we also find other works that we did not include in the data sources but they are relevant for our work. One of these works is [25] which proposes greater precision and efficiency by using specifically trained models, managing to reduce errors and improve efficiency in code generation. This work also has a greater scope by addressing a wider range of problems and scenarios, including those not typically considered critical but which can still benefit from optimization. These insights will not only boost productivity and precision in software development but also open new possibilities for innovation and complex problem-solving.

The authors of [26] propose the use of data contexts and advanced processing techniques to improve precision in generating programs from natural language descriptions. By filtering and sorting the most promising results, it makes it easier for users to quickly find the right program, reducing their dependence on deep technical knowledge.

[27] highlights the importance of conducting more comprehensive and rigorous evaluations of the code generated by LLMs, and indicates that the addition of automatically generated test cases significantly improves error detection. This demonstrates that positive results can be misleading due to insufficient testing. To fully trust the capabilities of LLMs in code generation, it is crucial to have a robust evaluation system that covers a wide range of cases and scenarios.

As we can see, the organization of the MAS varies between proposasls. One proposal is to organize the MAS as a set of agents, one pair of them (instructor and assistant) specialized in a phase of the traditional waterfall methodology of software development. The agents communicate through a sequence of intermediate chats dedicated to resolving specific tasks, called ”chat chain”. In each of these chats, an instructor leads with precise instructions, guiding the

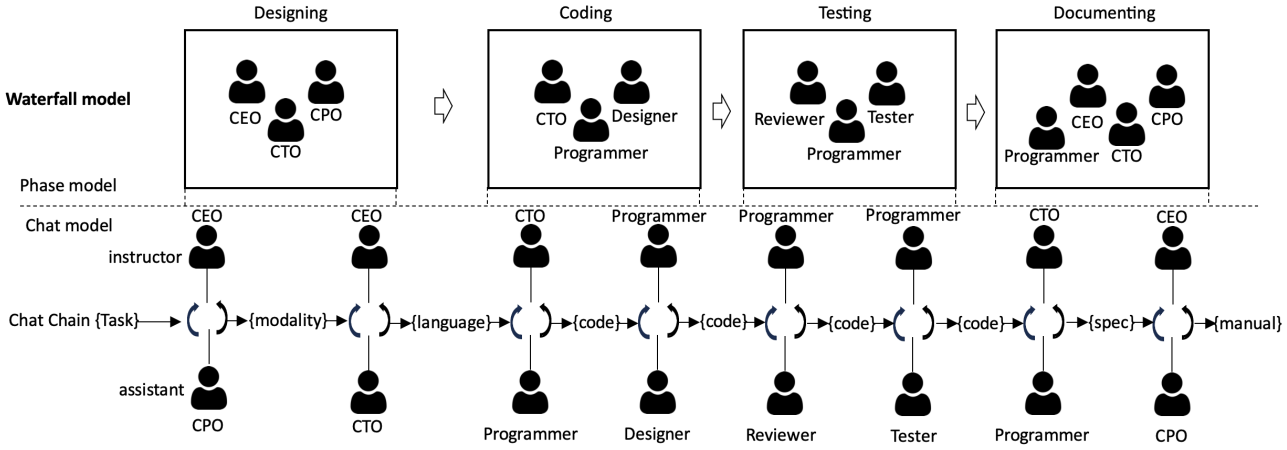

Fig. 3. Architecture composed of components at the phase level and at the chat level [9]

dialogue towards the successful completion of the task, while the assistant responds to these guidelines, provides relevant solutions and participates in critical discussions about its viability. On the other hand, it is possible to organize the MAS around general functions of a software development process. The user provides needs in form of prompts that are captured by the Task Agent, which creates tasks and adds them to the Task List Stack. The Prioritization Agent assigns priority to each task and sends them back to the Task List Stack. The Task Agent selects an incomplete task and assigns it to the Execution Agent, which retrieves context from memory and executes the task. Intermediate results are stored in memory until all tasks are completed. The final result is delivered to the User. Both approaches are valid, although the first one offers a clear and detailed view of the software development process, illuminating the decision-making path and providing opportunities for identifying and correcting errors. Hybrid approaches can also arise in the future.

The viability of this case study [9] depends on adequate technological availability, resources for development, tool maintenance, and the willingness of development teams to adopt a comprehensive user-centered approach. The result of this experiment focuses on knowing the operation, the compatibility between the tools and the possible points of improvement, since this can be a critical factor in the process, and its evolution depends on the constant growth of technologies.

E. Conclusions

The integration of LLMs into MAS means a revolutionary advance in the field of automatic code generation and collective intelligence for software development. The systematic review highlights significant progress in exploiting the adaptability and collaborative potential of MAS, augmented by the sophisticated language understanding capabilities of LLMs. This confluence has paved the way for efficient, scalable and intelligent code generation solutions.

Despite the benefits, the integration of LLMs into MAS presents several challenges, such as the complexity of implementation, ensuring accurate interpretation and application of standards, and managing the computational costs associated with sophisticated models like GPT-4.

In conclusion, the integration of LLMs with MAS represents a promising advancement in automatic code generation, offering significant potential for the future of software development. However, continuous refinement and rigorous evaluation are essential to address the inherent challenges and realize these technologies' capabilities. The insights from these studies provide a robust foundation for future research aimed at advancing the functionality and applications of LLM-based multi-agent systems.

IV. Research Method

To experiment and obtain answers useful for a future study, we used the MAS approach based on LLM for code generation. In our study, this method aims to identify the advantages and problems that this kind of system has in order to improve them in the future. It is worth noting that we did not make substantial changes to the proposed method. Generally, we decided to generate the classic arcade Snake video game. In this game, a snake moves across the screen collecting objects or food that appear in random locations. Each object consumed by the snake causes it to increase in size. Although there are different options for the case

study, this game was chosen because of its simplicity and to compare our results with those of [10]. The prompt we used was: "`generate a snake game in Python with a graphical user interface`".

To experiment and obtain insights useful for a future study, we employed the MAS approach based on LLM for code generation. Let us recall that it is not the purpose of this work to propose a new LLM-based MAS, but to identify the advantages and problems that this kind of system has in order to improve them in the future. For our experiment, we chose to generate the classic arcade snake video game. In this game, a snake moves across the screen, collecting objects or food that appear in random locations, with each object consumed causing the snake to increase in size. This game was selected due to its simplicity and the ability to compare our results with those of [10]. The prompt we used was: "`generate a snake game in Python with a graphical user interface`".

For the LMM-based MAS, we chose ChatDev [9], which adopts a sequential development model known as the waterfall model. This model includes four phases: design, coding, testing, and documentation, each of which must be completed before progressing to the next phase. As a result of the autonomous development process, ChatDev produces a requirements engineering specification, a software design plan, annotated software code, and an implementation manual, corresponding to each phase of the waterfall model.

ChatDev's approach seeks to solve code generation by incorporating the collaboration of multiple agents who play specific roles in each phase, such as programmers, reviewers, and testers, to facilitate a comprehensive and multidisciplinary development process. The dynamics of interaction between agents are structured through a sequence of communications in the form of a chat, breaking down each development phase into smaller and more manageable components called atomic subtasks (see Fig. 3). Each node within the chat chain represents a specific subtask, allowing at least two roles to participate in iterative and contextualized discussions. This interaction mechanism is designed to encourage the proposal and validation of solutions through constructive and focused dialogue. Consequently, the framework aims to optimize the delivery of the software product, ensuring its alignment with the expectations and needs of the end user.

In order to experiment and obtain answers that would be useful for a future study, we used the MAS approach based on LLM for code generation, so in our study this method has the objective of letting us know the necessary characteristics for code generation, it should be noted that we did not make substantial changes to the proposed method, in general, we decided to generate the classic arcade snake video game. In this game, a snake moves across the screen collecting objects or food that appear in random locations. Each object consumed by the snake causes it to increase in size. Although there are different options for the case study, this game was chosen because of its simplicity and to be able to compare our results with those of [10]. The proompt we use was: "`generate a snake game in Python with a graphical user interface`".

As LMM-based MAS we chose ChatDev [9], which adopts a sequential development model known as the waterfall model. This model has four different phases: design, coding, testing, and documentation, and each phase must be completed before moving on to the next. As a result of the autonomous development process, ChatDev produces a requirements engineering specification, a software design plan, an annotated software code and an implementation manual, each corresponding to each phase of the waterfall model.

ChatDev incorporates the collaboration of multiple agents who play specific roles in each phase, such as programmers, reviewers, and testers, to facilitate a comprehensive and multidisciplinary development process. The dynamics of interaction between agents are articulated through a sequence of communications in the form of a chat, decomposing each development phase into smaller and more manageable components, called atomic subtasks (see Fig. 3). In this context, each node within the chat chain symbolizes a specific subtask, allowing the participation of at least two roles in iterative and contextualized discussions. This interaction mechanism is designed to encourage the proposal and validation of solutions through constructive and focused dialogue. Therefore, the framework seeks to optimize the delivery of the software product, ensuring its alignment with the expectations and needs of the end user.

From a technical perspective, our setup of ChatDev employs the GPT-4 version of ChatGPT, utilizing resources provided by OpenAI [28]. These resources have furnished us with the necessary tools to conduct our study effectively and allowed us to explore our work in an appropriate manner. This configuration enables the adjustment of various parameters, such as setting the language model temperature to 0.2 to ensure controlled responses. During the coding phase, the setup permits up to five attempts to complete the code. The development uses Python 3.8 as the interpreter for testing.

TABLE III
EDUCATIONAL GAME EVALUATION MATRIX

Aspect to Evaluate	Evaluation Criteria
Design and Aesthetics	Does the game have an attractive design and an aesthetic consistent with the theme?
Functionality	Does the game function smoothly and without technical errors?
Educational Content	Does the game provide relevant educational content aligned with learning objectives?
Interactivity	Does the game allow for effective and stimulating interaction with the user?
Adaptability	Does the game adapt to the individual needs of the user?
Feedback	Does the game provide clear and constructive feedback to the user?
Originality	Does the game feature original elements or concepts in its design or content?

Using this setup, we tested the same prompt 10 times. The metrics of interest included the success rate (the percentage of attempts where the code was correctly generated), the number of lines of code generated, the time taken to generate the game, the number of generated tokens, and

the cost. Additionally, we subjectively evaluated the game in terms of design and aesthetics, functionality, educational content, interactivity, adaptability, feedback, and originality (see Table III).

V. Results

In this section, we present the results of our work that uses the capabilities of a Multi-Agent System composed by seven expert agents that autonomously generate code from high-level descriptions. These agents, each specialized in a different topic, collectively contribute to generating software output.

As a result of the development process of the snake game, agents broke down the specification into tasks to be addressed iteratively. This includes the elaboration of a software design plan, the writing of an initial code, the creation of test and implementation plans, and the generation of pertinent documentation. Within an iterative cycle, all the material produced was reviewed and tested, culminating in the creation of the game.

The process was done 10 times. We detected that, out of a total of 10 examples tried to be generated with the same prompt, there was an effectiveness rate of 70%. Three examples failed, presenting compilation errors. Therefore, 30% of the examples were incomplete. The left side of Fig. 4 displays the output with compilation errors, while the right side shows the running game output exhibiting the expected characteristics.

For the cases where there was a succesful output, the following average results were obtained. Game development was completed after five iterations, resulting in seven documents related to the initial message. The agents wrote 90 lines of code and prepared a manual that included 35 lines of information and 24 code statements. In total, 12,221 tokens were generated. A paid version of the GPT-4 model was used, which incurred an average cost of $0.02811 per game. Finally, it took the agents an average of 124 seconds to produce the final version of the game.

We can see in the Table IV that the following results were obtained. The various game examples were completed after five iterations, resulting in the necessary files being generated from the initial message. The agents wrote a minimum of 43 lines of code and a maximum of 116, they prepared 10 manuals for each example that included on average 30 lines of information and around 20 code statements necessary for the operation. In total, a minimum of 14,614 and a maximum of 31,352 tokens were generated. Finally, a paid version of the GPT-4 model was used, which generated an average cost of $0.0347684 per game. It took the agents an average of 156.8 seconds to produce the final version of the game.

Overall, the results demonstrate the potential of integrating MAS and LLMs for automatic code generation, showing significant improvements in development efficiency, productivity, and code quality. However, the presence of compilation errors in 30% of the cases indicates a need for further refinement and optimization of the system. Future research should focus on enhancing the robustness and adaptability of the agents to minimize errors and improve overall performance.

TABLE IV
Statistics of the different results obtained from software development.

Example	Code lines	Duration	Cost	Tokens
R1	69	187.00s	$0.042001	25533
R2	114	127.00s	$0.030817	18914
R3	116	207.00s	$0.050634	22796
R4	78	110.00s	$0.025490	11600
R5	98	120.00s	$0.027532	16932
R6	43	141.00s	$0.029454	17842
R7	94	169.00s	$0.033495	14614
R8	73	249.00s	$0.051203	31352
R9	79	134.00s	$0.028944	17745
R10	90	124.00s	$0.028114	17221

TABLE V
Educational Game Evaluation Matrix

Aspect to Evaluate	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Design and Aesthetics	0/10	9/10	0/10	8/10	9/10	8/10	8/10	0/10	8/10	7/10
Functionality	0/10	8/10	0/10	9/10	8/10	9/10	8/10	0/10	10/10	8/10
Educational Content	0/10	8/10	0/10	8/10	8/10	8/10	8/10	0/10	9/10	9/10
Interactivity	0/10	8/10	0/10	8/10	8/10	9/10	8/10	0/10	8/10	8/10
Adaptability	0/10	8/10	0/10	8/10	8/10	8/10	8/10	0/10	8/10	8/10
Feedback	0/10	8/10	0/10	8/10	8/10	8/10	8/10	0/10	8/10	8/10
Originality	0/10	8/10	0/10	8/10	8/10	8/10	8/10	0/10	8/10	8/10

We also evaluated the resulting software. Given that the product was a video game, we were inspired by the evaluation method described in [29]. Initially, we had to discard the three games with compilation errors. We then randomly selected one of the succesful programs and applied the evaluation method, aiming to measure the quality of the game. The aspects evaluated were Design and Aesthetics, Functionality, Educational Content, Interactivity, Adaptability, Feedback, and Originality. The results of this evaluation are presented in Table V. It should be noted that our objective was not to evaluate software as such; however, we aimed to gain insights that could serve as a reference for future research.

The evaluation of the generated game provides a perspective on the quality and potential of the multi-agent framework in automating code generation. The results indicate areas of strength, such as Educational Content and Originality, while also highlighting aspects that could benefit from further improvement, such as Functionality and Adaptability.

VI. Discussion

Our experiment allowed us to reinforce some of the key points identified in [10], but with some notes:

Increased Productivity. From a productivity standpoint, the results obtained allowed for the creation of a game example in an average time of 187.00s. Compared to traditional software development methods, this represents a

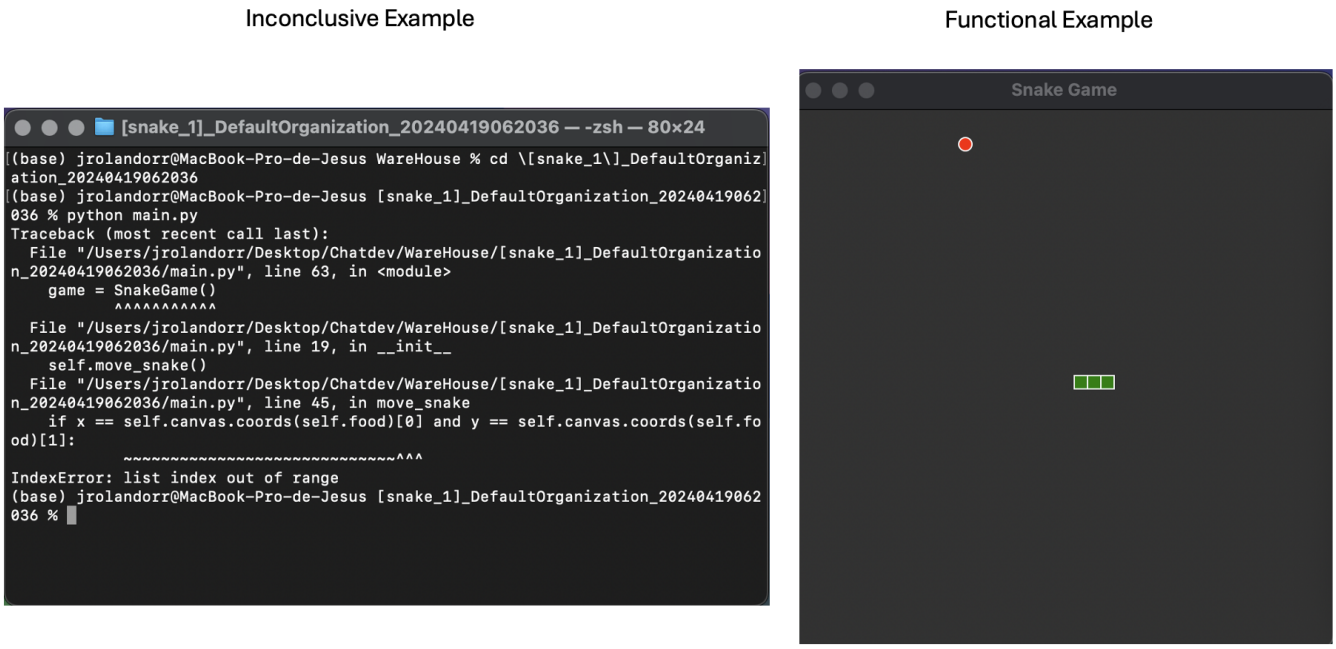

Fig. 4. Results obtained from the Multi agent systems for Software Development framework.

considerable productivity improvement.

Improved Quality of the Code. After manual inspection, we consider that acceptable code quality was obtained in most cases. One of the advantages we observed was that several agents reviewed the code and validated the team's work, reducing the probability of errors and improving code consistency. However, despite these filters, there were three cases of hallucinations where the code, although validated, was inconclusive. This suggests that code quality cannot always be guaranteed without a sufficiently detailed context or collaborative review.

Scalability and Adaptability. LLM-based MAS can be adapted to meet varying user needs. For instance, both AutoGPT and ChatDev feature modular structures that allow for quick adaptability to new requirements across different levels of complexity. However, these adaptations at the MAS level require specialized knowledge. At the agent level, the knowledge possessed by an LLM-based agent is typically general, necessitating fine-tuning for specific tasks and domains to achieve optimal results.

Simplified Code Debugging. The use of multiple agents makes the debugging process efficient, as different agents can detect and correct errors at various stages of development. In contrast to manual methods, which often lack multiple viewpoints and collaborative efforts, the multi-agent approach significantly enhances debugging efficiency. Our tests demonstrated that multiple agents verifying the generated code simplifies the developer's task when correcting final bugs to achieve functional code. However, it is important to note that this is not always the case; the effort required to correct issues when hallucinations appeared was not always justified.

Reduction of Human Error. One of the key advantages we observe is that automating development and dividing it among multiple autonomous agents significantly reduces the margin of human error in repetitive tasks. However, we have identified that more complex tasks can sometimes lead to hallucinations by the model when attempting to solve them. This issue represents one of the primary limitations of the current system.

Continuous Learning and Adaptation. One of the key points we could reinforce is that both approaches are based on continuous evolution. This capacity for constant learning and adaptation not only optimizes the immediate efficiency of the system, but also increases its effectiveness in the long term. As agents accumulate experience and adjust their behavior, they become more precise and effective in performing complex tasks. This ensures that the system not only responds adequately to current needs, but is also prepared to face future challenges with a solid foundation of knowledge and improved skills.

VII. Conclusions

This study investigates the feasibility and potential of integrating Multi-Agent Systems (MAS) with Large Language Models (LLMs) to address the limitations of traditional manual code creation methods. As we saw, a MAS can exploit the specialized capabilities of different agents, facilitating the management of complex tasks and quality control in software development. LLM-based agents have the ability

to correctly respond in a constrained scenario such as the snake game generation consider in this paper, but as we discuss, hallucinations can appear and in that case the agent, and the whole system, fails. Our research demonstrates that MAS and LLMs can collaboratively enhance software development by improving productivity, adaptability, and code quality. The integration of these technologies facilitates automatic code generation and opens new avenues for AI-based software engineering.

Future research should focus on overcoming the limitations identified in the current integration of LLM and multi-agent systems. This includes improving the understanding and processing capabilities of agents within dynamic and complex environments, refining the system's ability to handle ambiguous instructions more effectively, and reducing the incidence of code hallucinations. Furthermore, exploring the application of these systems in other domains of software engineering, such as its integration with model-driven engineering, can also be of interest.

REFERENCES

[1] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Foundations and Trends® in Programming Languages*, vol. 4, no. 1-2, pp. 1–119, 2017. [Online]. Available: http://dx.doi.org/10.1561/2500000010

[2] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2023.

[3] A. Kanade, P. Maniatis, G. Balakrishnan, and K. Shi, "Learning and evaluating contextual embedding of source code," *arXiv preprint arXiv:2001.00059*, 2020.

[4] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," *arXiv preprint arXiv:2305.02309*, 2023.

[5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, and Y. Burda, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[7] D. Huang, Q. Bu, J. M. Zhang, M. Luck, and H. Cui, "Agentcoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2023.

[8] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[9] C. Qian, X. Cong, W. Liu, C. Yang, W. Chen, Y. Su, Y. Dang, J. Li, J. Xu, D. Li, Z. Liu, and M. Sun, "Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.

[10] Z. Rasheed, M. Waseem, K.-K. Kemell, W. Xiaofeng, A. N. Duc, K. Systä, and P. Abrahamsson, "Autonomous agents in software development: A vision paper," 2023.

[11] C. Zhang, K. Yang, S. Hu, Z. Wang, G. Li, Y. Sun, C. Zhang, Z. Zhang, A. Liu, S.-C. Zhu, X. Chang, J. Zhang, F. Yin, Y. Liang, and Y. Yang, "Proagent: Building proactive cooperative agents with large language models," *arXiv preprint arXiv:2308.11339*, 2024.

[12] C. Qian, Y. Dang, J. Li, W. Liu, W. Chen, C. Yang, Z. Liu, and M. Sun, "Experiential co-learning of software-developing agents," *arXiv preprint arXiv:2312.17025*, 2023.

[13] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, "Synchromesh: Reliable code generation from pre-trained language models," *arXiv preprint arXiv:2201.11227*, 2022.

[14] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[15] J. Chen, H. Guo, K. Yi, B. Li, and M. Elhoseiny, "Visualgpt: Data-efficient adaptation of pretrained language models for image captioning," *arXiv preprint arXiv:2102.10407*, 2022.

[16] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering – a systematic literature review," *Information and Software Technology*, vol. 51, no. 1, pp. 7 – 15, 2009, special Section - Most Cited Articles in 2002 and Regular Research Papers. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0950584908001390

[17] K. Petersen, R. Feldt, S. Mujtaba, and M. Mattsson, "Systematic mapping studies in software engineering," in *Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE'08. BCS Learning and Development Ltd., 2008, p. 68–77.

[18] N. S. M. Yusop, J. Grundy, and R. Vasa, "Reporting usability defects: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 43, no. 9, pp. 848–867, 2017.

[19] U. Dissemination, "The database of abstracts of reviews of effects (dare)," 12 2002.

[20] S. Suri, S. Das, K. Singi, K. Dey, V. Sharma, and V. Kaulgud, "Software engineering using autonomous agents: Are we there yet?" in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 1855–1857. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ASE56229.2023.00174

[21] A. Barcaui and A. Monat, "Who is better in project planning?generative artificial intelligence or project managers?" *Project Leadership and Society*, vol. 4, p. 100101, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666721523000224

[22] M. Viana, P. Alencar, E. Guimarães, E. Cirilo, and C. Lucena, "Creating a modeling language based on a new metamodel for adaptive normative software agents," *IEEE Access*, vol. 10, pp. 13 974–13 996, 2022.

[23] S. Hu, T. Huang, F. Ilhan, S. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," in *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. Los Alamitos, CA, USA: IEEE Computer Society, nov 2023, pp. 297–306. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TPS-ISA58951.2023.00044

[24] OpenAI, "Chatgpt: Chat generative pre-trained transformer," https://www.openai.com/chatgpt, 2023, accessed: 2024-08-05.

[25] G. Prasath and S. Karande, "Synthesis of mathematical programs from natural language specifications," 2023.

[26] A. Khatry, J. Cahoon, J. Henkel, S. Deep, V. Emani, A. Floratou, S. Gulwani, V. Le, M. Raza, S. Shi, M. Singh, and A. Tiwari, "From words to code: Harnessing data for program synthesis from natural language," 2023.

[27] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," 2023.

[28] OpenAI, "Gpt-4: Generative pre-trained transformer 4," https://www.openai.com, 2023, accessed: 2023-10-01.

[29] L. Geovanny, "Rúbrica de evaluación para juegos educativos," https://edtk.co/rbk/24189, 2024, last acces [04/20/24].